SCIKIT-LEARN MODEL SELECTION & EVALUATION

모델 선택, 검증 및 평가지표 핵심 정리

교차 검증(3.1) · 하이퍼파라미터 튜닝(3.2) · 임계값 최적화(3.3) · 평가지표(3.4) · 검증 곡선(3.5) 요약 및 코드

3.1. Cross-validation: evaluating estimator performance (교차 검증: 모델 성능 평가)

데이터 누수(Data Leakage)를 방지하고 모델의 일반화 성능을 객관적으로 측정하기 위한 기법들을 다룹니다.

3.1.1. Computing cross-validated metrics (교차 검증 지표 계산)

- `cross_val_score`: 단일 평가 지표로 교차 검증 점수를 산출합니다.
- `cross_validate`: 다중 평가 지표 지원 및 학습/예측 소요 시간, 학습 점수까지 함께 반환합니다.
- `cross_val_predict`: 각 폴드(Fold)의 테스트 데이터에 대한 예측값을 결합하여 반환합니다 (성능 평가보다는 시각화나 스태킹 앙상블의 입력 피처 생성용으로 사용).

3.1.2. Cross validation iterators (교차 검증 분할기)

데이터 특성	주요 분할기 클래스	핵심 특징 및 적용 목적
i.i.d. (독립 동일 분포)	`KFold`, `RepeatedKFold`, `LeaveOneOut` (LOO), `LeavePOut` (LPO), `ShuffleSplit`	데이터 샘플 간 독립성이 보장될 때 사용하는 기본 분할 및 무작위 반복 분할 기법
클래스 불균형 (Stratified)	`StratifiedKFold`, `StratifiedShuffleSplit`	각 폴드의 타깃 클래스 비율을 원본 데이터와 동일하게 유지하여 편향 방지
그룹화된 (Grouped)	`GroupKFold`, `StratifiedGroupKFold`, `LeaveOneGroupOut`, `GroupShuffleSplit`	동일 환자·동일 사용자 등 그룹 단위로 데이터가 쪼개져 훈련/검증 세트에 동시에 들어가는 것을 방지
시계열 (Time Series)	`TimeSeriesSplit`	과거 데이터로 학습하고 미래 데이터로 검증하도록 시간 순서를 유지하며 분할

3.1.3. Data leakage (데이터 누수 방지)

- 스케일링, 결측치 대체 등의 전처리는 반드시 교차 검증 분할 내부에서 수행해야 합니다.
- `Pipeline` 또는 `make_pipeline`을 사용하여 전처리와 모델 학습을 하나로 묶어 교차 검증에 전달하는 것이 표준 방식입니다.

```python
from sklearn.model_selection import StratifiedKFold, cross_validate,
cross_val_predict
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# 전처리와 모델을 파이프라인으로 결합하여 데이터 누수 방지
pipe = make_pipeline(StandardScaler(), LogisticRegression())
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# 다중 지표 평가 및 학습 점수/소요 시간 반환
cv_res = cross_validate(
    pipe, X, y, cv=skf,
    scoring=['accuracy', 'f1', 'roc_auc'],
    return_train_score=True
)
oof_preds = cross_val_predict(pipe, X, y, cv=skf)  # Out-of-fold 예측값 생성
```

3.2. Tuning the hyper-parameters of an estimator (하이퍼파라미터 튜닝)

모델이 데이터로부터 직접 학습하지 않는 설정값(하이퍼파라미터)의 최적 조합을 찾는 방법입니다.

- **3.2.1. Exhaustive Grid Search (`GridSearchCV`):** 지정한 하이퍼파라미터 후보들의 모든 조합(Cartesian product)을 전수 조사하여 최고 성능의 파라미터를 탐색합니다.
- **3.2.2. Randomized Parameter Optimization (`RandomizedSearchCV`):** 파라미터 분포에서 정해진 횟수(`n_iter`)만큼 무작위로 샘플링하여 탐색합니다. 탐색 공간이 넓을 때 `GridSearchCV`보다 훨씬 효율적으로 최적해에 근접할 수 있습니다.
- **3.2.3. Successive Halving (`HalvingGridSearchCV`, `HalvingRandomSearchCV`):** 초기에는 적은 데이터(리소스)로 많은 후보군을 평가한 뒤, 상위 성능 후보만 추려 리소스를 늘려가며 반복 평가하는 토너먼트 방식의 고속 탐색 기법입니다.
- **3.2.4. Tips for parameter search (파라미터 탐색 팁):** `scoring` 파라미터로 최적화 기준 지표 지정, `refit=True` 설정 시 최적 파라미터로 전체 학습 데이터 자동 재학습, `n_jobs=-1` 설정으로 CPU 코어 병렬 활용이 가능합니다.

```
from sklearn.experimental  enable_halving_search_cv
 sklearn.model_selection  GridSearchCV, RandomizedSearchCV, HalvingRandomSearchCV
 sklearn.ensemble  RandomForestClassifier
 scipy.stats  randint

rf = RandomForestClassifier(random_state=42)
param_dist = {: randint(50, 300), : [3, 5, 10, ]}


rand_search = RandomizedSearchCV(
    rf, param_distributions=param_dist, n_iter=20,
    scoring=, cv=5, refit=, n_jobs=-1, random_state=42
).fit(X_train, y_train)


halving_search = HalvingRandomSearchCV(
    rf, param_distributions=param_dist, factor=3,
    scoring=, cv=5, n_jobs=-1, random_state=42
).fit(X_train, y_train)import#
noqafromimportfromimportfromimport'n_estimators''max_depth'None# 무작위 탐색 (넓은 탐색 공간에 적합)'f1'True# 연속 반감법 기반 고속 토너먼트 탐색'f1'
```

3.3. Tuning the decision threshold for class prediction (분류 예측을 위한 결정 임계값 튜닝)

확률 기반 분류기에서 기본 임계값(0.5) 대신 비즈니스 목적이나 불균형 데이터에 맞는 최적의 임계값(Threshold)을 찾는 기법입니다.

- **3.3.1. Post-hoc tuning of the decision threshold (`TunedThresholdClassifierCV`):** 모델의 예측 확률(`predict_proba`)이나 결정 함수(`decision_function`) 출력값에 대해, 지정된 평가 지표(예: F1-score, Precision, Recall 등)를 극대화하는 최적의 임계값을 교차 검증으로 탐색합니다.
- **3.3.2. `FixedThresholdClassifier`:** 도메인 지식이나 비용 행렬(Cost Matrix)을 통해 이미 알고 있는 특정 임계값을 고정하여 분류기에 적용할 때 사용합니다.

```
from sklearn.model_selection import TunedThresholdClassifierCV,
FixedThresholdClassifier

# 1) 교차 검증으로 F1 점수를 극대화하는 최적 임계값 자동 탐색
tuned_clf = TunedThresholdClassifierCV(
    estimator=pipe, scoring='f1', cv=5
).fit(X_train, y_train)
print("최적 임계값:", tuned_clf.best_threshold_)

# 2) 도메인 지식으로 정해진 특정 임계값(예: 0.3) 고정 적용
fixed_clf = FixedThresholdClassifier(estimator=pipe, threshold=0.3).fit(X_train,
y_train)
```

3.4. Metrics and scoring: quantifying the quality of predictions (평가지표 및 스코어링)

예측 결과의 품질을 정량화하는 API 규칙과 문제 유형별 주요 지표를 다룹니다.

3.4.1. The scoring parameter (스코어링 파라미터 규칙)

- 문자열 이름(예: `'accuracy'`, `'f1'`, `'neg_mean_squared_error'`)을 통해 교차 검증 및 하이퍼파라미터 탐색 함수에 지표를 전달합니다.
- 관례상 모든 스코어링 함수는 **"클수록 좋은 값(Higher return values are better)"**을 따르므로, 오차(Error) 지표는 음수(`neg_`)가 붙습니다.
- `make_scorer`를 이용해 사용자 정의 평가 함수를 스코어러로 변환할 수 있습니다.

문제 유형	세부 구분	주요 Scikit-learn 평가지표 함수
3.4.2. 분류 (Classification)	이진/다중 분류 공통	`accuracy_score`, `balanced_accuracy_score`, `precision_score`, `recall_score`, `f1_score`, `confusion_matrix`, `classification_report`
	확률 기반 지표	`roc_auc_score`, `average_precision_score` (PR AUC), `log_loss` (Cross-entropy), `brier_score_loss`
3.4.3. 회귀 (Regression)	오차 기반	`mean_absolute_error` (MAE), `mean_squared_error` (MSE), `root_mean_squared_error` (RMSE), `mean_absolute_percentage_error` (MAPE)
	설명력 기반	`r2_score` (결정계수 R^2), `explained_variance_score`
3.4.4. 군집화 (Clustering)	정답 레이블 있음	`adjusted_rand_score` (ARI), `normalized_mutual_info_score` (NMI), `v_measure_score`
	정답 레이블 없음	`silhouette_score` (실루엣 계수), `calinski_harabasz_score`, `davies_bouldin_score`

```
from sklearn.metrics import make_scorer, fbeta_score, root_mean_squared_error,
silhouette_score

# 사용자 정의 평가 함수를 스코어러로 변환 (Recall 가중치를 높인 F2 스코어)
f2_scorer = make_scorer(fbeta_score, beta=2)

# 오차 지표는 스코어링 문자열에서 neg_ 접두어 사용
cv_rmse = cross_val_score(reg_model, X, y, cv=5,
scoring='neg_root_mean_squared_error')
```

3.5. Validation curves: plotting scores to evaluate models (검증 곡선: 모델 평가를 위한 점수 시각화)

모델의 과소적합(Underfitting)과 과대적합(Overfitting)을 진단하기 위해 점수를 시각화하는 도구입니다.

- **3.5.1. Validation curve (`validation_curve`, `ValidationCurveDisplay`):** 단일 하이퍼파라미터 값을 변화시키면서 훈련 점수와 검증 점수의 추이를 비교합니다. 파라미터가 너무 단순해 둘 다 낮으면 **과소적합**, 훈련 점수만 높고 검증 점수가 떨어지면 **과대적합**으로 진단합니다.
- **3.5.2. Learning curve (`learning_curve`, `LearningCurveDisplay`):** 훈련 데이터 샘플 수를 점진적으로 늘려가면서 훈련 점수와 검증 점수의 변화를 측정합니다. 데이터를 더 수집하는 것이 성능 향상에 도움이 될지(**분산 감소**), 아니면 모델 자체의 복잡도를 높여야 할지(**편향 감소**) 판단하는 근거로 활용합니다.

```
import numpy as np
from sklearn.model_selection import ValidationCurveDisplay, LearningCurveDisplay
from sklearn.tree import DecisionTreeClassifier

tree = DecisionTreeClassifier(random_state=42)

# 1) 검증 곡선: max_depth 변화에 따른 과소/과대적합 진단
ValidationCurveDisplay.from_estimator(
    tree, X, y, param_name='max_depth', param_range=np.arange(1, 15),
    cv=5, scoring='accuracy', n_jobs=-1
)

# 2) 학습 곡선: 샘플 수 증가에 따른 편향-분산 진단
LearningCurveDisplay.from_estimator(
    tree, X, y, train_sizes=np.linspace(0.1, 1.0, 10),
    cv=5, scoring='accuracy', n_jobs=-1
)
```

실무 적용 요약: 전처리와 모델은 반드시 `Pipeline`으로 묶어 데이터 누수를 차단하고, 불균형·그룹·시계열 등 데이터 구조에 맞는 CV 분할기를 선택한 뒤 `RandomizedSearchCV` (또는 `HalvingRandomSearchCV`)와 `TunedThresholdClassifierCV`로 모델 및 의사결정 기준을 최적화합니다.